

Weekly 09



SISTEMAS DISTRIBUIDOS

Sergio Luis Angel Romero

DOCENTE: JESUS ARIEL

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA

CORPORACION UNIVERSITARIA DEL HUILA
Vignada Minieducación

ING. EN SISTEMAS

NEIVA - HUILA

2026

Tabla de Contenido

Tabla de Contenido	2
1. Descripción del Proyecto.....	3
2. Listado de ADR Identificados.....	3
ADR 1 – Arquitectura en Capas	3
Identificación del patrón	3
Estructura del proyecto:.....	4
Análisis del proyecto	4
Antipatrón identificado	4
Segmento de código del proyecto	5
Diseño e implementación del ADR.....	6
Impacto sobre el sistema	7
ADR 5 – Uso de Docker para Contenerización	7
Identificación del patrón	7
Análisis del proyecto	7
Antipatrón identificado	7
Segmento de código del proyecto	7
Diseño e implementación del ADR.....	8
Impacto sobre el sistema	9
ADR 6 – Uso de Maven para Gestión de Dependencias	9
Identificación del patrón	9
Análisis del proyecto	9
Antipatrón identificado	9
Segmento de código del proyecto	10
Diseño e implementación del ADR.....	10
Impacto sobre el sistema	10

INFORME – ARCHITECTURE DECISION RECORD (ADR)

Proyecto: Gestor de Pedidos en Spring Boot.

1. Descripción del Proyecto

El sistema Gestor de Pedidos es una aplicación backend desarrollada con Spring Boot que permite gestionar información de clientes y pedidos mediante una API REST. El sistema está organizado en diferentes capas: Controller, Service, Repository y Model. Además utiliza Spring Data JPA para persistencia, Maven para gestión de dependencias y Docker para contenerización.

2. Listado de ADR Identificados

ADR 1 – Arquitectura en Capas (Controller – Service – Repository)

ADR 2 – Uso de Spring Data JPA para persistencia

ADR 3 – Uso de API REST para comunicación

ADR 4 – Inyección de dependencias con Spring

ADR 5 – Uso de Docker para contenerización

ADR 6 – Uso de Maven para gestión de dependencias

ADR 1 – Arquitectura en Capas

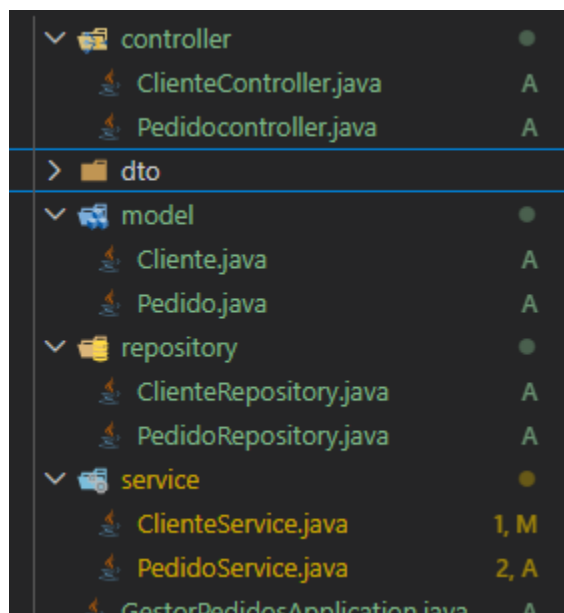
Identificación del patrón

El sistema implementa una arquitectura en capas, separando las responsabilidades en distintos componentes del sistema:

- Controller
- Service
- Repository
- Model

Esta estructura permite organizar el código de manera clara y facilita el mantenimiento del sistema.

Estructura del proyecto:



Análisis del proyecto

El sistema Gestor de Pedidos debe permitir gestionar información de clientes y pedidos. Para ello es necesario:

- recibir solicitudes HTTP
- procesar lógica de negocio
- acceder a la base de datos

Si todas estas funciones se implementaran en una sola clase, el código sería difícil de mantener y escalar.

Por esta razón se decidió implementar una arquitectura en capas.

Antipatrón identificado

Un antipatrón sería mezclar todas las responsabilidades dentro de un controlador.

Ejemplo incorrecto:

```

7 main / java / com / sergio / gestor_pedidos / model / Cliente.java / ...
1 @RestController
2 public class PedidoController {
3
4     @Autowired
5     PedidoRepository pedidoRepository;
6
7     @PostMapping("/pedido")
8     public Pedido crearPedido(@RequestBody Pedido pedido){
9         return pedidoRepository.save(pedido);
10    }
11
12 }
13

```

Problemas:

- mezcla lógica de negocio
- mezcla acceso a datos
- genera código difícil de mantener

Segmento de código del proyecto

Controller:

```

@RestController
@RequestMapping("/clientes")
@RequiredArgsConstructor
public class ClienteController {

    private final ClienteService clienteService;

    @PostMapping
    public Cliente crear(@RequestBody Cliente cliente) {
        return clienteService.crearCliente(cliente);
    }

    @GetMapping
    public List<Cliente> listar() {
        return clienteService.listarClientes();
    }
}

```

Service:

```
import java.util.List;

@Service
@RequiredArgsConstructor
public class ClienteService {

    private final ClienteRepository clienteRepository;

    // Esta función es para la creación del cliente
    public Cliente crearCliente(Cliente cliente) {
        return clienteRepository.save(cliente);
    }

    //Esta función es para el listado de clientes :3
    public List<Cliente> listarClientes() {
        return clienteRepository.findAll();
    }
}
```

Repository:

```
main > java > com > sergio > gestor_pedidos > repository > ClienteRepository.java > { } com.sergio

package com.sergio.gestor_pedidos.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.sergio.gestor_pedidos.model.Cliente;

public interface ClienteRepository extends JpaRepository<Cliente, Long> {
}
```

Esto demuestra que cada capa tiene una responsabilidad específica.

Diseño e implementación del ADR

Se decidió implementar una arquitectura en capas, donde:

Controller → maneja las solicitudes HTTP

Service → contiene la lógica de negocio

Repository → gestiona la persistencia de datos

Model → representa las entidades del sistema

Esta decisión mejora la organización del proyecto.

Impacto sobre el sistema

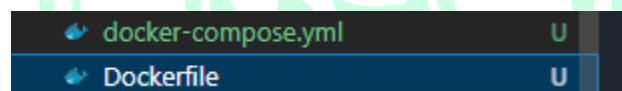
Beneficios:

- mejor organización del código
- mayor facilidad de mantenimiento
- separación clara de responsabilidades
- facilita futuras ampliaciones del sistema

ADR 5 – Uso de Docker para Contenerización

Identificación del patrón

El proyecto utiliza Docker para contenerizar la aplicación, permitiendo ejecutar el sistema en diferentes entornos sin depender de configuraciones locales.



Análisis del proyecto

Durante el desarrollo de aplicaciones es común que el sistema funcione en una máquina pero falle en otra debido a diferencias de configuración.

Para evitar este problema se decidió utilizar contenedores Docker.

Antipatrón identificado

Un antipatrón sería depender únicamente del entorno local de cada desarrollador.

Problemas:

- diferentes versiones de Java
- diferentes configuraciones
- problemas al desplegar el sistema

Segmento de código del proyecto

Ejemplo del archivo Dockerfile (de momento):


```

application.properties A  Cliente.java A  Dockerfile U
Dockerfile > ...
1  FROM eclipse-temurin:21-jdk
2
3  WORKDIR /app
4
5  COPY target/gestor_pedidos.jar app.jar
6
7  EXPOSE 8080
8
9  ENTRYPOINT ["java", "-jar", "app.jar"]

```

Ejemplo de docker-compose.yml:

```

application.properties A  Cliente.java A  docker-compose.yml U X
docker-compose.yml
1  version: "3.9"
2
3  > Run All Services
4  services:
5
6  > Run Service
7  postgres:
8    image: postgres:15
9    container_name: postgres_gestor_pedidos
10   restart: always
11   environment:
12     POSTGRES_DB: gestor_pedidos
13     POSTGRES_USER: postgres
14     POSTGRES_PASSWORD: 1234
15   ports:
16     - "5432:5432"
17   volumes:
18     - postgres_data:/var/lib/postgresql/data
19
20 > Run Service
21 app:
22   build: .
23   container_name: gestor_pedidos_app
24   depends_on:
25     - postgres
26   ports:
27     - "8080:8080"

```

Diseño e implementación del ADR

Se decidió utilizar Docker para empaquetar la aplicación junto con todas sus dependencias.

Esto permite ejecutar el sistema mediante contenedores.

Impacto sobre el sistema

Beneficios:

- portabilidad entre entornos
- facilidad de despliegue
- consistencia en desarrollo y producción
- simplificación del proceso de ejecución

ADR 6 – Uso de Maven para Gestión de Dependencias

Identificación del patrón

El proyecto utiliza Apache Maven como herramienta de gestión de dependencias y construcción del proyecto.



Análisis del proyecto

El sistema utiliza múltiples librerías externas como:

- Spring Boot
- Spring Data JPA
- Dependencias para REST

Gestionar estas librerías manualmente sería complejo.

Por esta razón se decidió utilizar Maven.

Antipatrón identificado

Un antipatrón sería descargar manualmente las librerías necesarias.

Problemas:

- errores en versiones
- dificultad para actualizar dependencias
- falta de control del proyecto

Segmento de código del proyecto

Ejemplo del archivo pom.xml:

```

<dependencies>
  <!-- Web (Spring MVC + Tomcat embebido) -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- JPA / Hibernate -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- Driver PostgreSQL (tu application.properties está
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>

```

Esto permite que Maven descargue automáticamente la dependencia necesaria.

Diseño e implementación del ADR

Se decidió utilizar Maven como herramienta de construcción del proyecto.

Maven permite:

- gestionar dependencias
- compilar el proyecto
- ejecutar pruebas
- generar el paquete final de la aplicación

Impacto sobre el sistema

Beneficios:

- gestión automática de dependencias
- facilidad para compilar el proyecto
- control de versiones de librerías
- estandarización del proceso de desarrollo

enlace de git: <https://github.com/sangel3232/Gestor-de-Pedidos-Portal/tree/dev>

enlace notion:

<https://www.notion.so/903ee3f7a312830c8fe081f1669e485b?v=a37ee3f7a3128363bba208761ca0ae40>



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación